UD1. Introducción a Android Studio y Jetpack Compose

0. Índice

1. Introducción
2. Jetpack Compose
3. Columnas
4. Filas
5. Text
6. Box

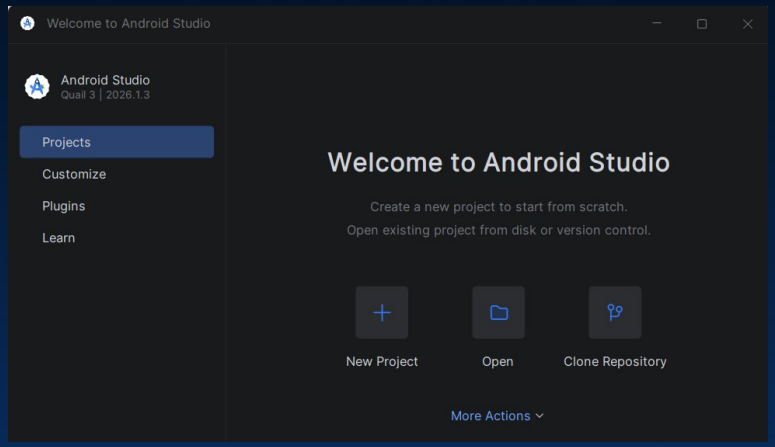
1. Introducción

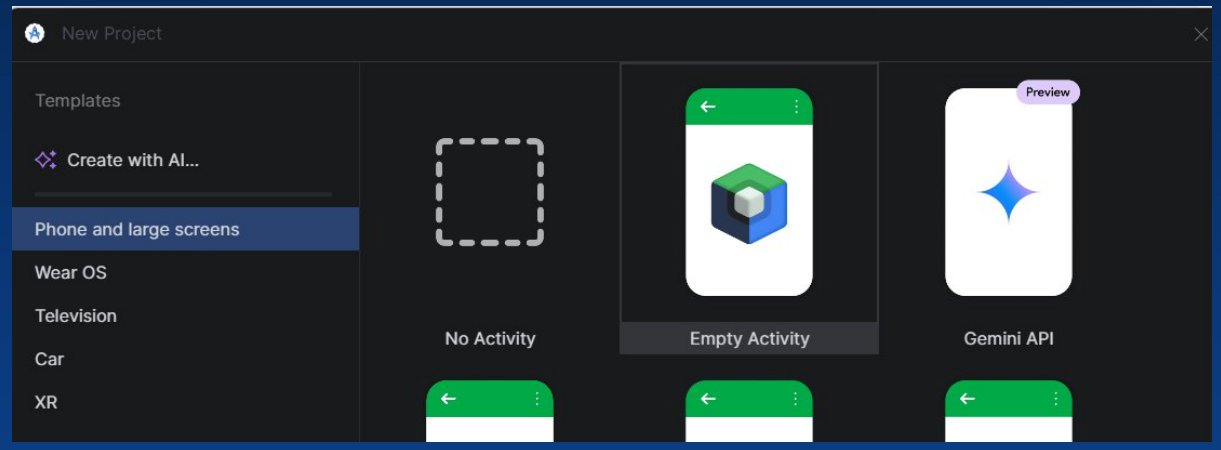
- En este primer tema, daremos los primeros pasos para comenzar a desarrollar las interfaces de Android.
- Con Android Studio y Jetpack Compose, crearás tus primeras interfaces y aprenderás a utilizar las herramientas.
- Empieza a conocer la Web Android Developers: https://developer.android.com/?hl=es-419
- Visita el canal de Aristi Dev lleno de videos interesantes y sencillos: https://www.youtube.com/@AristiDevs
- Comienza por el video de 9 horas…

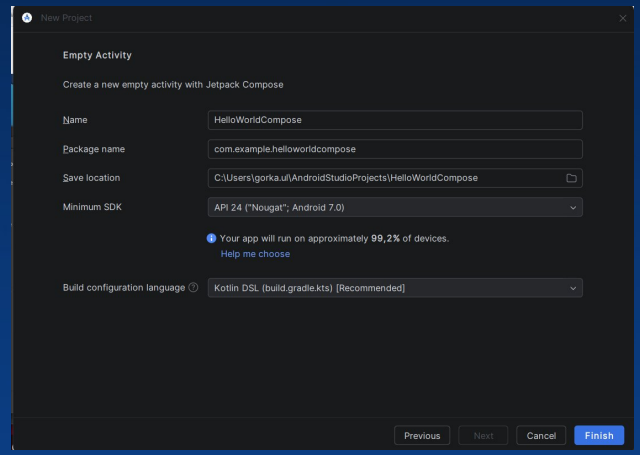


1. Introducción

- Para crear un proyecto ve a Projects —> New Project
- Seleccionamos Empty Activity
- Nombramos la actividad

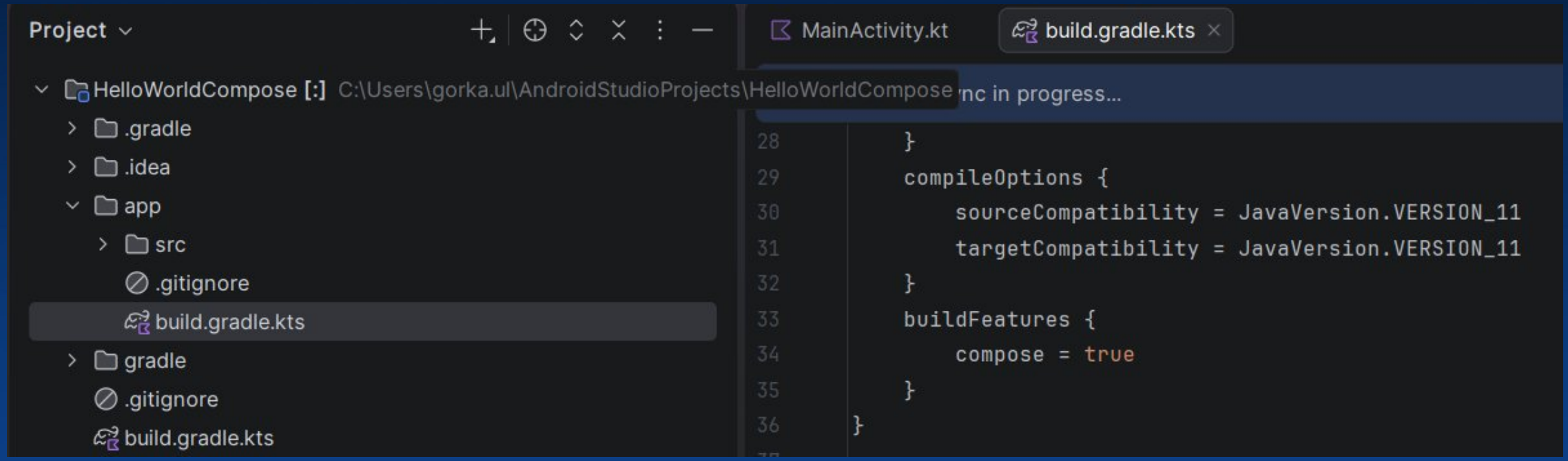






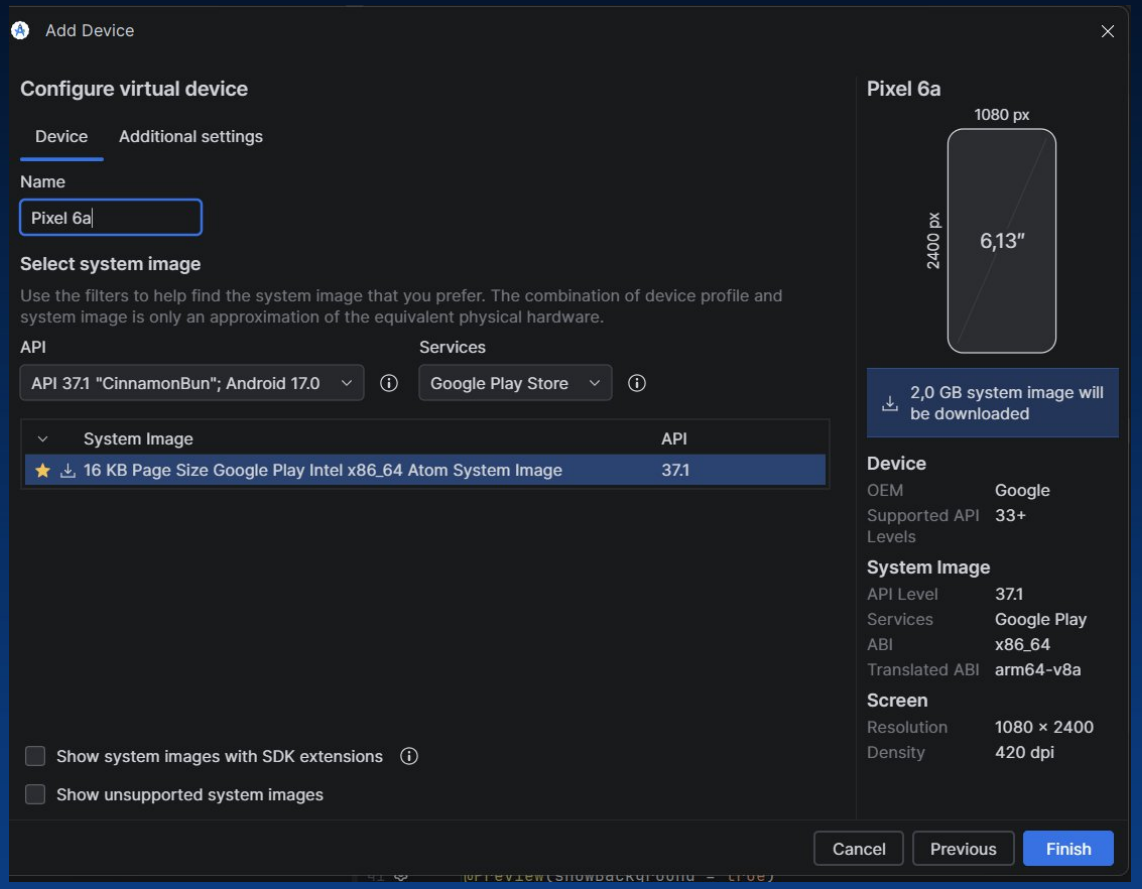
1. Introducción

- En las versiones más recientes Jetpack Compose viene instalado, para asegurarte de sí está en uso mira en el archivo build.gradle.kts y busca en el código la opción compose = true



1. Introducción

- Necesitamos crear antes de ejecutar un programa un device ya sea real (un móvil que tengamos) o virtual. Para ello vamos a Tools —> Device Manager.
- Pulsamos el botón + y elegimos un Virtual Device.
- A la hora de elegir, es conveniente si queremos universalidad elegir un número bajo por ejemplo (Pixel 6a) y que tenga el símbolo de google Play Store para evitar problemas con requerimientos futuros.



2. Jetpack Compose

- Jetpack Compose es un kit de herramientas moderno para crear IU nativas en Android. Jetpack Compose simplifica y acelera el desarrollo de IU en Android con menos código, herramientas potentes y API intuitivas kotlin.

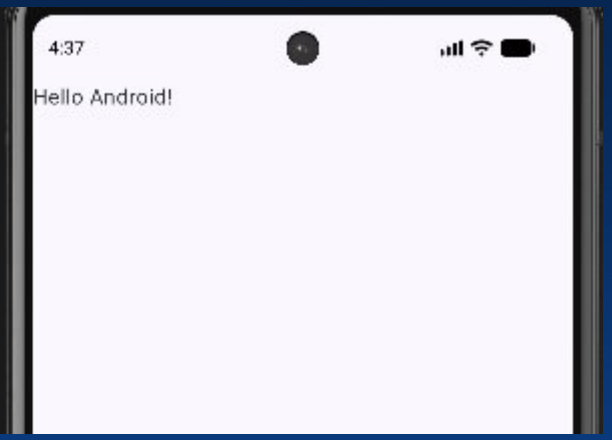
- https://developer.android.com/compose



2. Jetpack Compose

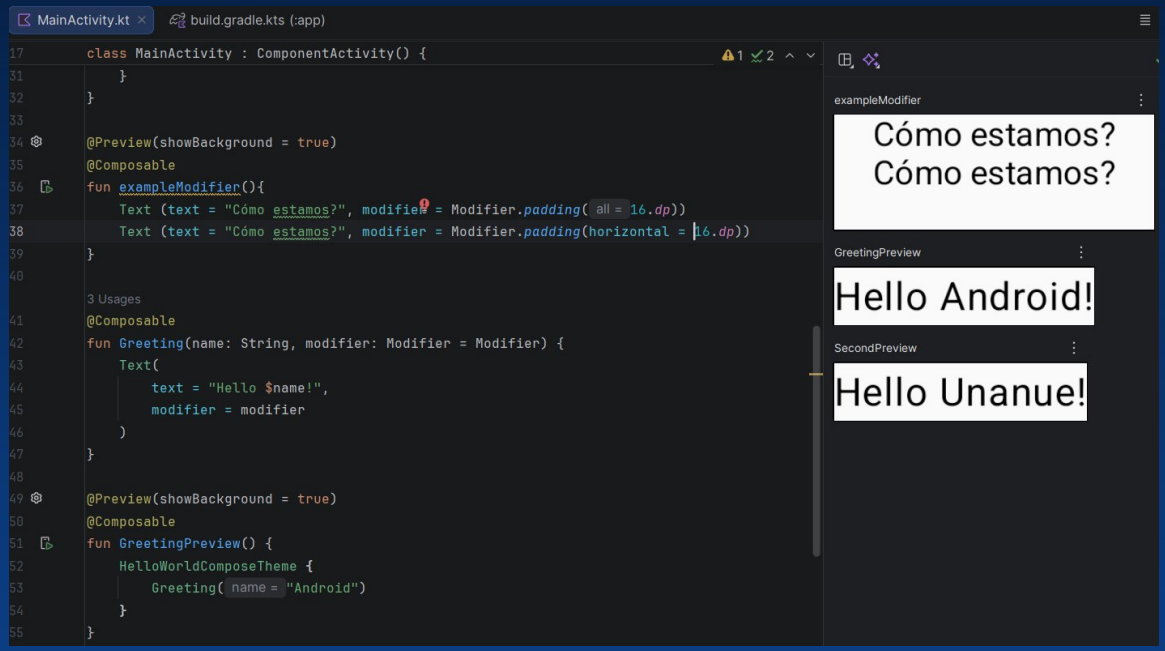
- Toda función que queramos que Jetpack Compose la reconozca tiene que tener la etiqueta @Composable

```
@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {
    Text(
        text = "Hello $name!",
        modifier = modifier
    )
}
```



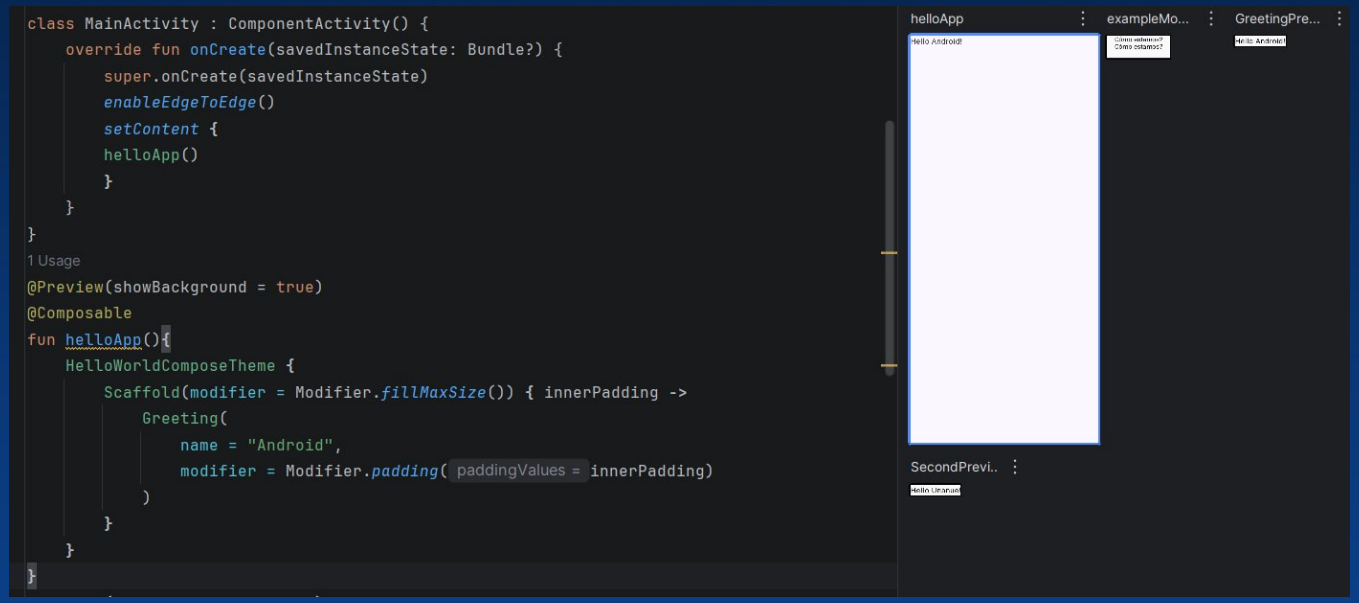
2. Jetpack Compose

- La función Text además de mostrar un texto, puede ser editada con marametros con la siguiente sintaxis:
  - Modifier = Modifier.padding(16.dp)
  - Modifier = Modifier.padding(horizontal = 16.dp)



2. Jetpack Compose

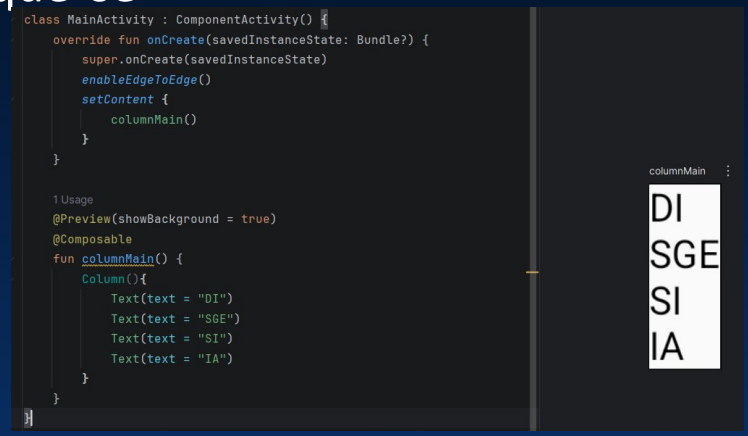
- Si creamos una función helloApp y metemos dentro el contenido que había en setContent de la función onCreate y lo sustituimos por la llamada a la función, conseguimos visualizar en el preview una interfaz que ocupa toda la pantalla.



3. Columnas

- Las columnas permiten ordenar los elementos verticalmente sin que se superpongan.

```
Column (modifier = Modifier.fillMaxSize()) {
        ...
}
```



- Se pueden concatenar modificadores escribiendo . entre ellos.

```
Column (modifier = Modifier.fillMaxSize()).background(Color.Red)){
        ...
}
```

3. Columnas

- Forma correcta de trabajar con colores: Existe en nuestro proyecto en la ruta res —> values —> colors.xml un archivo .xml que contiene todos los colores que se usarán en la aplicación, con la intención de crear una paleta de marca de producto (https://paletton.com/#uid=1000u0kllllaFw0g0qFqFg0w0aF) y no tener que recordar los códigos hexadecimales.

Column(modifier = Modifier.fillMaxSize().background(colorResource(id = R.color.miColorRojo))){



3. Columnas

- Insertar imágenes:

  Image(painter = painterResource(id = R.drawable.male), contentDescription = "logo hombre")

- Se pueden insertar imagenes bitmap y vectoriales, siendo estas últimas muy beneficiosas para reescalarlas sin pixelación y para cambiar su color de forma fácil.

```
    }

    1 Usage
    @Preview(showBackground = true)
    @Composable
    fun columnMain() {
        Column(modifier = Modifier.fillMaxSize().background(colorResource(id = R.color.miColorRojo))) {
            Image(painter = painterResource(id = R.drawable.male), contentDescription = "logo hombre")
            Text(text = "DI")
            Text(text = "SGE")
            Text(text = "SI")
            Text(text = "IA")
        }
    }
}
```



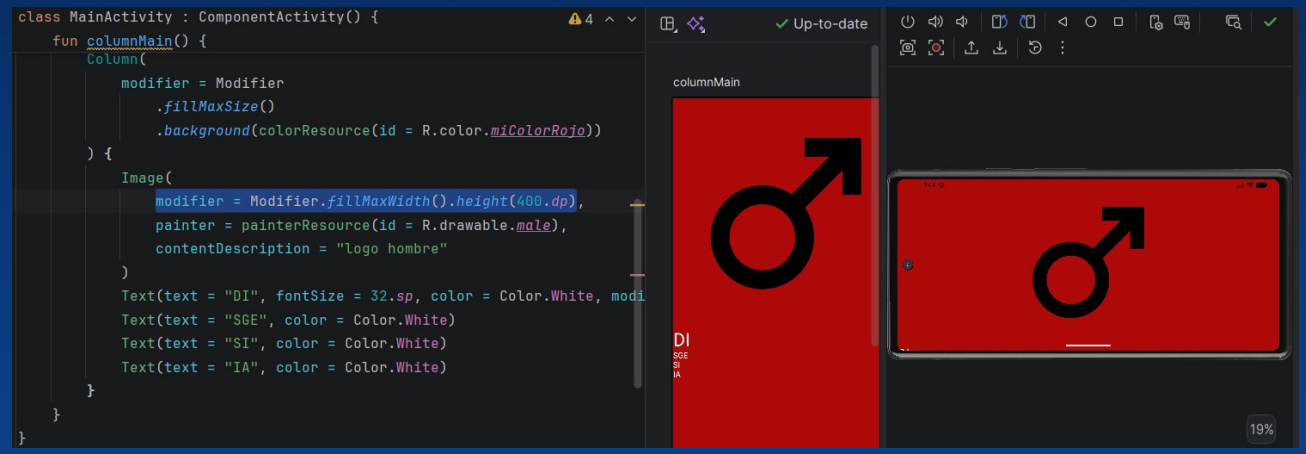
3. Columnas

- Más modificadores

  Text(text = "DI", fontSize = 32.sp, color = Color.White, modifier = Modifier.fillMaxWidth())

- Si añadimos el siguiente modificador a la imagen, cuando giramos la pantalla no se ven el resto de textos, por lo que tendremos que añadir un scroll.

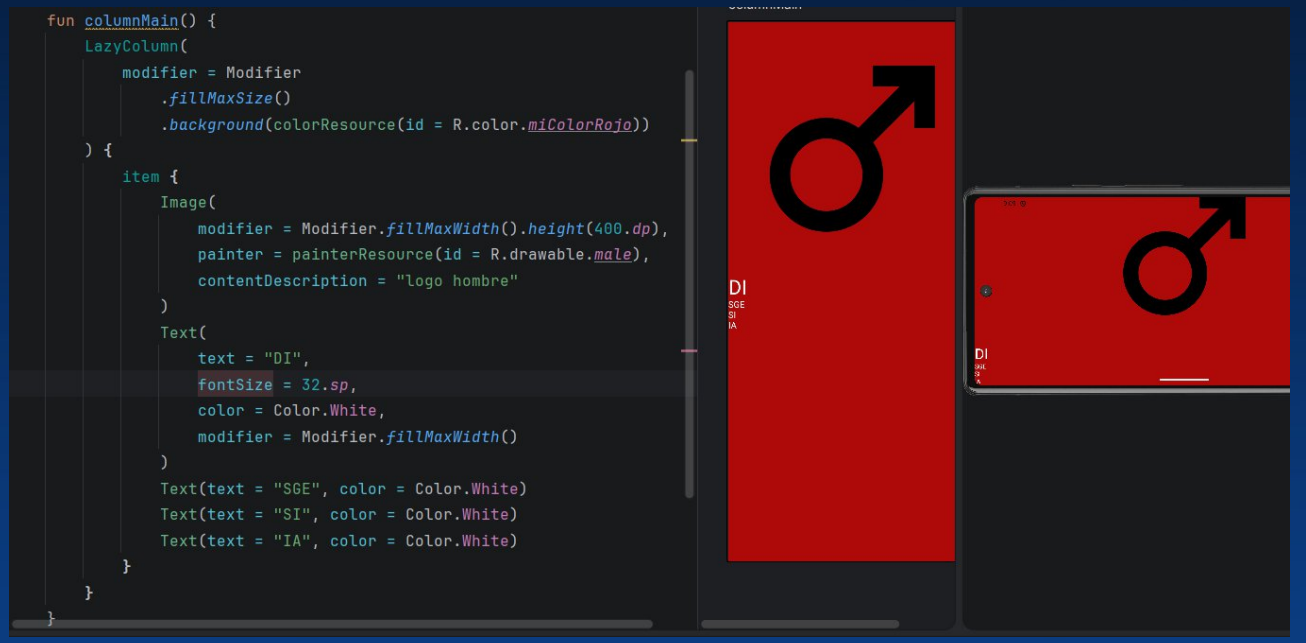
  modifier = Modifier.fillMaxWidth().height(400.dp)



3. Columnas

- Para añadir funcionalidad de scroll sustituimos nuestro Column por LazyColum, además debemos meter todos nuestros elementos dentro de un item:

```
fun columnMain() {
        LazyColumn(
            modifier = Modifier
                .fillMaxSize()
                .background(colorResource(id = R.color.miColorRojo))
        ) {
            item {
                Image(
                    ...
                )
                Text(
                    ...
                )
                Text(text = "SGE", color = Color.White)
                Text(text = "SI", color = Color.White)
                Text(text = "IA", color = Color.White)
            }
        }
```



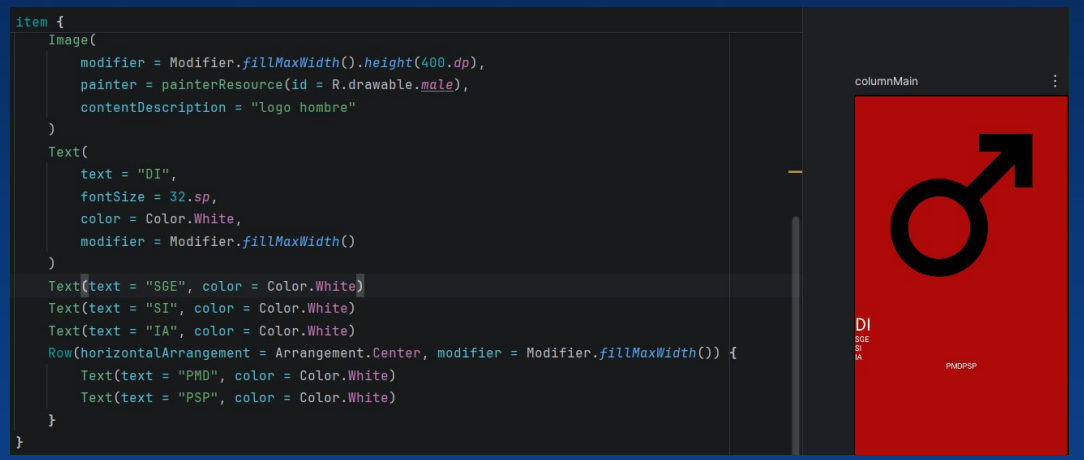
4. Filas

- Las filas nos permiten ordenar elementos de manera horizontal. La interfaz no puede empezar con más de una fila o columna, es decir, tenemos que tener una fila o columna inicial y dentro encadenar todas las que queramos. Su sintaxis es:

```
Row(horizontalArrangement = Arrangement.Center, modifier = Modifier.fillMaxWidth()) {
   Text(text = "PMD", color = Color.White)
   Text(text = "PSP", color = Color.White)
}
```

- Si queremos dejar el mismo espacio entre elementos:

```
horizontalArrangement = Arrangement.SpaceBetween
```



4. Filas

- El modificador padding añade un margen interior al elemento:

```
Row(horizontalArrangement = Arrangement.Center, modifier =
Modifier.fillMaxWidth()).padding(16.dp)) {
   Text(text = "PMD", color = Color.White)
   Text(text = "PSP", color = Color.White)
}
```

- Si queremos dejar el mismo espacio entre elementos:

```
horizontalArrangement = Arrangement.SpaceBetween
```



4. Filas

- Para hacer scroll horizontal usamos LazyRow:

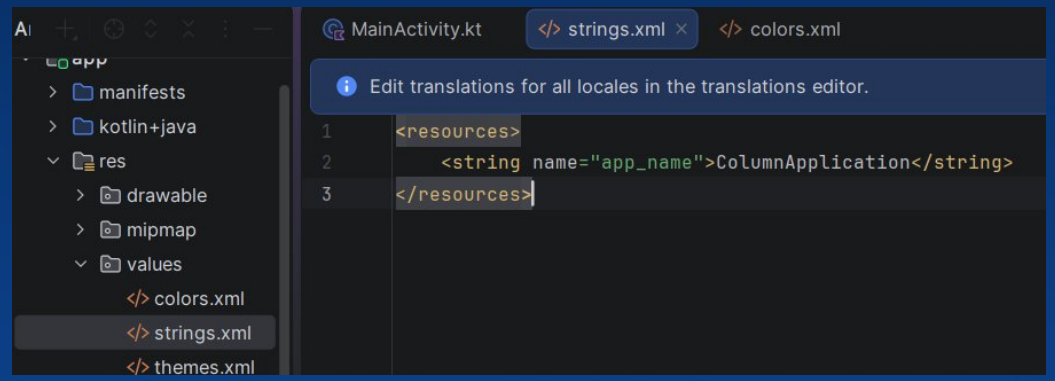
```
LazyRow(horizontalArrangement = Arrangement.SpaceBetween, modifier =
Modifier.fillMaxWidth().padding(16.dp)) {
   item{
      Text(text = "PMD", color = Color.White)
      Text(text = "PSP", color = Color.White)
      Text(text = "PMD", color = Color.White)
      Text(text = "PSP", color = Color.White)
      Text(text = "PMD", color = Color.White)
      Text(text = "PSP", color = Color.White)
      Text(text = "PMD", color = Color.White)
      Text(text = "PSP", color = Color.White)
      Text(text = "PMD", color = Color.White)
      Text(text = "PSP", color = Color.White)
      Text(text = "PMD", color = Color.White)
      Text(text = "PSP", color = Color.White)
      Text(text = "PMD", color = Color.White)
      Text(text = "PSP", color = Color.White)
   }
}
```

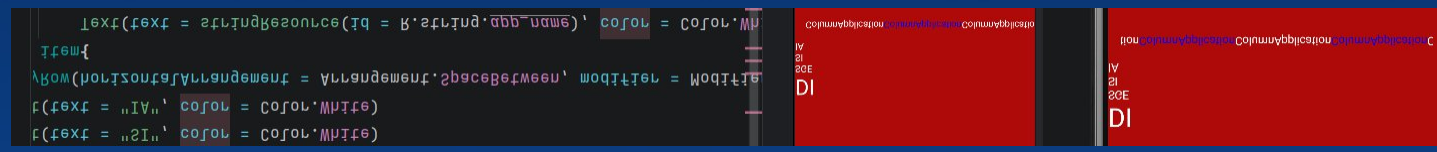


4. Filas

- String.xml: Cuando desarrollamos una aplicación, es común que se realicen cambios en los nombres que aparecen en pantalla como textos o botones, y si además optamos por hacer nuestra aplicación en diferentes idiomas, cambiar cada nombre de cada ítem se vuelve una tarea tediosa. Por eso, en vez de escribir cada texto directamente como hasta ahora, vamos a escribirlos en el archivo res —> values —> string.xml. De esa manera será más cómodo y fácil realizar cualquier cambio posterior.

Text(text = stringResource(id = R.string.app_name), color = Color.White)

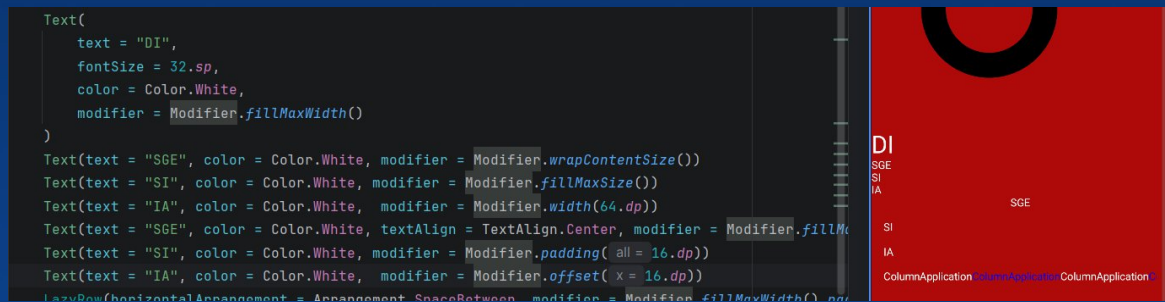




5. Text

- Otros modificadores:
  - Modifier.wrapContentSize: Hace que el elemento Text ocupe el tamaño del texto que contenga.
  - Modifier.fillMaxSize o fillMaxWidth / fillMaxHeight: Hace que el elemento Text ocupe el máximo tamaño en ambas dimensiones o en una concreta.
  - Modifier.width / height: Especifica el tamaño en dp.
  - textAlign = TextAlign.Center: Hace que esté alineado al centro o izquierda, derecha, arriba o abajo. Tiene que ir obligatoriamente con un fillMax.
  - Modifier.padding: Crea un margen dentro de la caja del ítem Text.
  - Modifier.offset: Crea un margen o desplazamiento fuera de la caja del Text.

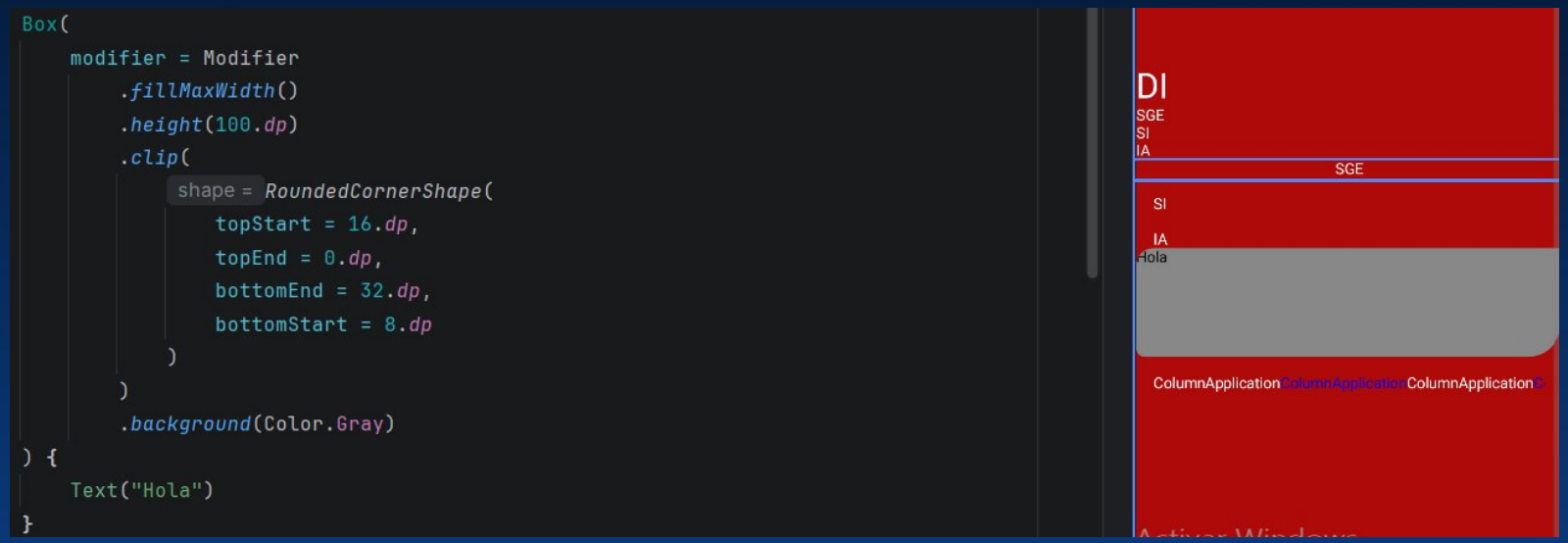
  - Si se quiere aplicar el mismo modificador a todos los elementos seguramente tendrás una manera de hacerlo a su elemento padre, como la columna o la fila.



6.Box

- La caja nos permite agregar un fondo de un tamaño concreto y agrupar dentro distintos elementos:

```
Box(
    modifier = Modifier
        .fillMaxWidth()
        .height(100.dp)
        .clip(
            RoundedCornerShape(
                topStart = 16.dp,
                topEnd = 0.dp,
                bottomEnd = 32.dp,
                bottomStart = 8.dp
            )
        )
        .background(Color.Gray)
) {
    Text("Hola")
}
```



- Ejercicio 1